

**ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ РЕСПУБЛИКИ
МОЛДОВА**

**ФАКУЛЬТЕТ МАТЕМАТИКИ И ИНФОРМАТИКИ
ДЕПАРТАМЕНТ ИНФОРМАТИКИ**

Кроитор Александр

Автономная операционная система самооптимизации и самолечения

ПРИКЛАДНАЯ ИНФОРМАТИКА

Курсовая работа

Директор Департамента:	Carcelea Titu, doctor, conferențiar universitar
------------------------	--

Научный руководитель:	Nichita Nartea
-----------------------	----------------

Автор:	Кроитор Александр, IA2403
--------	---------------------------

КИШИНЁВ – 2026

СОДЕРЖАНИЕ

СПИСОК ПРИНЯТЫХ СОКРАЩЕНИЙ	3
ВВЕДЕНИЕ	4
I ТЕОРЕТИЧЕСКАЯ БАЗА	6
1.1 Проблема дрейфа состояния	6
1.2 Декларативное управление состоянием	7
1.3 Промежуточное представление	7
1.4 Граф ресурсов	8
1.5 Реактивная модель	9
1.6 Акторы и отказоустойчивость	11
1.7 Транзакционность и компенсация	11
1.8 Проверяемость	11
II АРХИТЕКТУРА FROGOS	13
2.1 Общий конвейер	13
2.2 Организация исходного кода	14
2.3 Акторная модель и Supervisor	16
2.4 DSL	16
2.5 IR	17
2.6 Toad	17
2.7 Planner	18
2.8 Executor	18
2.9 Runtime FrogOS	19
2.10 Расширяемость доменов	19
2.11 Поколения и аудит	20
2.12 Граница привилегий	21
2.13 Наблюдаемость runtime	21
2.14 Требования к системе	21
2.14.1 Функциональные требования	21
2.14.2 Нефункциональные требования	22

2.15	Сравнительный анализ	23
2.15.1	Сравнение с NixOS	23
2.15.2	Сравнение с GNU Guix System	24
2.15.3	Сравнение с традиционными дистрибутивами . .	25
2.15.4	Сравнение с самовосстанавливающимися платформами	25
III	ПРАКТИЧЕСКАЯ ЧАСТЬ	26
3.1	Структура выполненной реализации	26
3.2	Реализация DSL и IR	26
3.3	Реализация наблюдения состояния	30
3.4	Реализация Supervisor	30
3.5	Реализация планирования	31
3.6	Примеры графов состояния	31
3.7	Реализация исполнения и отката	32
3.8	Проверка свойств	33
3.9	Реализация dry-run пути	34
3.10	End-to-end окружение	35
3.11	Инструменты разработки	36
	ЗАКЛЮЧЕНИЕ	38
	БИБЛИОГРАФИЯ	40

СПИСОК ПРИНЯТЫХ СОКРАЩЕНИЙ

Сокращение	Расшифровка
API	Программный интерфейс приложения.
CPU	Центральный процессор; в работе также используется для обозначения метрик загрузки процессора.
DSL	Предметно-ориентированный язык.
EDSL	Встроенный предметно-ориентированный язык.
IR	Промежуточное представление, используемое как контракт между подсистемами.
FFI	Интерфейс вызова функций между языками программирования.
JSON	Текстовый формат обмена структурированными данными.
CLI	Интерфейс командной строки.
CI	Непрерывная интеграция.
FHS	Стандарт иерархии файловой системы Unix-подобных систем.
MAPE-K	Модель автономного управления Monitor, Analyze, Plan, Execute over Knowledge.
mpsc	Модель канала “many producers, single consumer”: много отправителей, один получатель.
protobuf	Protocol Buffers, бинарный формат сериализации структурированных данных.

ВВЕДЕНИЕ

Современная операционная система всё чаще рассматривается не только как набор пакетов и служб, но и как управляющая система, обязанная поддерживать заданное состояние в изменяющейся среде. Администратор задаёт намерение: какие пакеты должны быть доступны, какие модули должны работать, какие пользователи должны существовать и какие реакции должны следовать из наблюдаемого состояния. Однако обычные дистрибутивы Linux в основном предоставляют инструменты настройки, а не замкнутый контур самокоррекции.

Тема работы — автономная операционная система самооптимизации и самолечения. В рамках данной работы FrogOS рассматривается как реактивная модель операционной системы. Система получает декларативное намерение, компилирует его в проверенное промежуточное представление, наблюдает текущее состояние хоста, строит план перехода и применяет этот план с возможностью отката. Работа опирается на идеи декларативных системных конфигураций NixOS и GNU Guix, но переносит их в модель runtime-согласования [3, 5].

Решаемая проблема состоит в дрейфе состояния Linux-системы. После ручных изменений, сбоя службы, удаления пакета или изменения профиля питания фактическое состояние перестаёт совпадать с тем, что описано администратором. В традиционной модели это обнаруживается внешним мониторингом или ручной проверкой. FrogOS переносит эту задачу внутрь runtime: система хранит проверенное намерение, наблюдает хост и строит план восстановления согласованности. Такая постановка близка к controller-подходу Kubernetes, где система сравнивает желаемое состояние с наблюдаемым и выполняет действия согласования [10].

Цель работы состоит в исследовании архитектуры FrogOS и формализации её как системы управления состоянием:

$$I \times S_t^{obs} \rightarrow P_t \rightarrow S_{t+1}^{obs},$$

где I — проверенное намерение, S_t^{obs} — наблюдаемое состояние в момент времени t , P_t — план изменения системы, а S_{t+1}^{obs} — новое состояние после применения плана.

Важно, что S_t^{obs} в этой модели означает наблюдаемое состояние, а не полное истинное состояние машины. FrogOS не имеет прямого доступа ко всем фактам

о хосте. Она работает с тем, что было собрано Toad через probe-компоненты, и поэтому каждый новый цикл начинается с повторного наблюдения.

Для достижения цели работа строится как последовательная реализация системы:

- определить модель декларативного намерения и правила его проверки;
- спроектировать IR как стабильный контракт между конфигурацией, планировщиком и runtime;
- реализовать наблюдение состояния хоста через Toad и привести наблюдения к единому снимку *SystemState*;
- построить Planner, который преобразует пару *IRDocument, SystemState* в детерминированный план перехода;
- реализовать Executor для последовательного применения шагов плана и выполнения undo-операций при ошибке;
- связать DSL, IR, Toad, Planner, Executor и верхний runtime FrogOS в базовый dry-run путь применения конфигурации;
- подготовить интеграционное окружение, в котором компоненты FrogOS можно собирать и проверять вместе без изменения основной системы разработчика.

I ТЕОРЕТИЧЕСКАЯ БАЗА

Теоретическая база работы связывает три идеи: декларативное описание системы, наблюдение текущего состояния и вычисление плана согласования. FrogOS не пытается угадать “истинное” состояние мира. Система работает только с наблюдаемым снимком S_t^{obs} , полученным от probe-компонентов, и поэтому все гарантии планирования относятся к паре “проверенное намерение — наблюдаемое состояние”. Такая постановка опирается на идеи *autonomic computing*, где самоуправляемая система получает высокоуровневые цели и сама выполняет конфигурацию, оптимизацию, восстановление и защиту [1]. Архитектурно эта идея раскрыта в IBM blueprint через контур MAPE-K и разделение управляемого ресурса, точек управления, автономного менеджера и базы знаний [2]. В терминах MAPE-K FrogOS реализует практический вариант цепочки Monitor, Analyze, Plan, Execute: Toad отвечает за наблюдение, Planner — за анализ и план, Executor — за применение, а IR и поколения выступают базой знаний контура.

1.1. Проблема дрейфа состояния

Linux-система редко остаётся в том состоянии, в котором её однажды настроил администратор. Пакет может быть удалён вручную, служба может завершиться с ошибкой, конфигурационный файл может быть изменён сторонним инструментом, а пользователь может появиться или исчезнуть вне основного процесса управления. Такое расхождение между ожидаемым и фактическим состоянием называется дрейфом состояния.

Дрейф опасен тем, что он накапливается незаметно. Отдельное изменение может не сломать систему сразу, но через некоторое время администратор уже не может точно сказать, какие действия привели к текущему состоянию. При восстановлении после сбоя это создаёт проблему: нужно не только знать, что произошло, но и понимать, какое состояние должно считаться правильным.

FrogOS рассматривает эту проблему как задачу постоянного согласования. В системе есть проверенное намерение администратора и есть наблюдаемое состояние хоста. Если эти две части расходятся, система должна построить понятный план возврата к требуемому состоянию. В этом смысле FrogOS ближе к модели управляющего контура, чем к обычному набору команд настройки.

Похожая идея используется в Kubernetes controller pattern: контроллер сравнивает желаемое состояние с наблюдаемым и выполняет действия, приближающие систему к желаемому состоянию [10].

1.2. Декларативное управление состоянием

Декларативная модель отличается от императивной тем, что пользователь описывает результат, а не последовательность команд. В императивной модели администратор пишет: установить пакет, создать пользователя, запустить службу. В декларативной модели он пишет: пакет должен присутствовать, пользователь должен существовать, служба должна быть активной.

Это различие существенно для повторного применения. Императивный скрипт может быть опасно запускать дважды, если он не был специально написан идемпотентным. Декларативная система, наоборот, должна стремиться к тому, чтобы повторное применение уже достигнутого состояния не создавало новых действий. Если пакет уже установлен, он не устанавливается повторно; если модуль уже активен, шаг включения не нужен.

В FrogOS декларативное намерение отделяется от плана исполнения. DSL описывает, что должно быть истинно для системы. Planner решает, какие действия нужны именно сейчас, потому что этот ответ зависит от наблюдаемого состояния. Один и тот же IR может дать пустой план на уже согласованной машине и непустой план на машине после дрейфа. Такой подход родственен декларативной конфигурации NixOS и GNU Guix System, но переносит вычисление изменения в runtime-контур наблюдения [3, 5].

1.3. Промежуточное представление

Промежуточное представление нужно для того, чтобы разделить язык пользователя и runtime. DSL может быть удобным и типобезопасным, но runtime не должен зависеть от внутренних деталей построителя конфигурации. Поэтому результатом компиляции становится IRDocument: стабильный документ, который можно сохранить, сравнить, передать планировщику и использовать в поколениях.

IR выполняет роль контракта. Если схема IR меняется, это должно быть явно видно по версии документа. Planner может отвергнуть неподдерживаемую

версию, а не пытаться интерпретировать неизвестные поля. Такой подход делает ошибку совместимости ранней и понятной.

Важное свойство IR состоит в том, что он хранит намерение, а не команды исполнения. Например, профиль может содержать действие включения модуля и политику выбора порта, но он не содержит системного вызова, shell-команды или конкретной последовательности операций backend. Это позволяет менять runtime без переписывания пользовательской конфигурации.

1.4. Граф ресурсов

Состояние операционной системы удобно описывать как граф ресурсов. Вершинами являются управляемые сущности: пользователи, пакеты, модули, сетевые правила, профиль питания. Рёбра показывают зависимость или отношение между ресурсами: пользователь может иметь пользовательские пакеты, модуль может требовать открытый порт, конфигурация модуля может требовать установленного пакета.

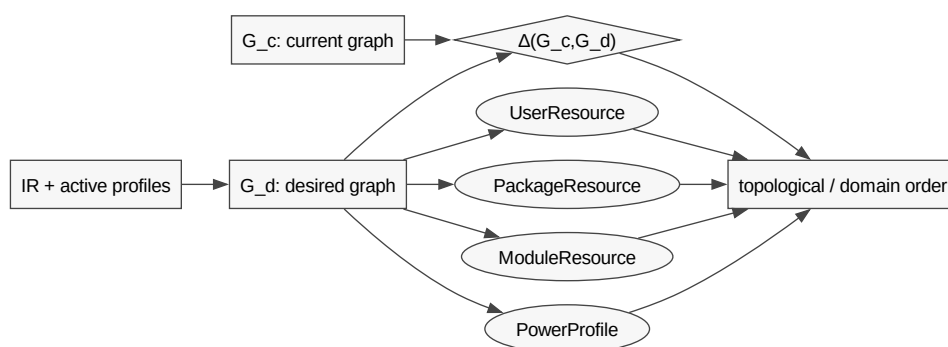


Рис. 1: Обобщённая модель графа ресурсов FrogOS

Planner строит два графа. Первый граф отражает текущее состояние, полученное от Toad. Второй граф отражает требуемое состояние, полученное из IR и активных профилей. Затем Planner сравнивает эти графы и оставляет только отличающиеся части. Так появляется план: не полный список всех ресурсов, а набор действий, который переводит систему из текущего состояния в требуемое.

Графовая модель полезна тем, что она делает порядок применения не случайным. Пользователи создаются до пользовательских пакетов, пакеты устанавливаются до активации модулей, а сетевые правила применяются до

запуска сервиса, которому нужен порт. Часть порядка задаёт FrogOS, часть делегируется специализированным backend: Nix управляет внутренними зависимостями пакетов, dinit управляет зависимостями служб.

Формально один цикл согласования можно описать так:

$$I \xrightarrow{\text{compile}} IR, \quad (IR, S_t^{obs}) \xrightarrow{\text{graph}} (G_d, G_c), \quad P_t = \text{plan}(G_d \setminus G_c),$$

$$S_{t+1}^{obs} = \text{observe}(\text{apply}(P_t, S_t^{obs})).$$

Здесь I — пользовательское намерение, IR — проверенный документ, S_t^{obs} — наблюдаемое начальное состояние, G_d — требуемый граф, G_c — граф, построенный из наблюдений, P_t — план перехода, а S_{t+1}^{obs} — следующий наблюдаемый снимок после применения плана и повторного наблюдения. Эта запись намеренно использует S^{obs} , а не S : Planner не имеет доступа к абсолютному состоянию хоста, он работает только с данными, которые смогли собрать probe-компоненты.

1.5. Реактивная модель

Реактивная система работает не как одноразовый инсталлятор, а как контур, который реагирует на изменения входных данных. В FrogOS такими входами являются проверенный IR и снимок SystemState. Изменился IR — нужно пересчитать план. Изменилось состояние хоста — тоже нужно пересчитать план. Если план пустой, система ничего не делает. Если план непустой, Executor получает шаги и применяет их.

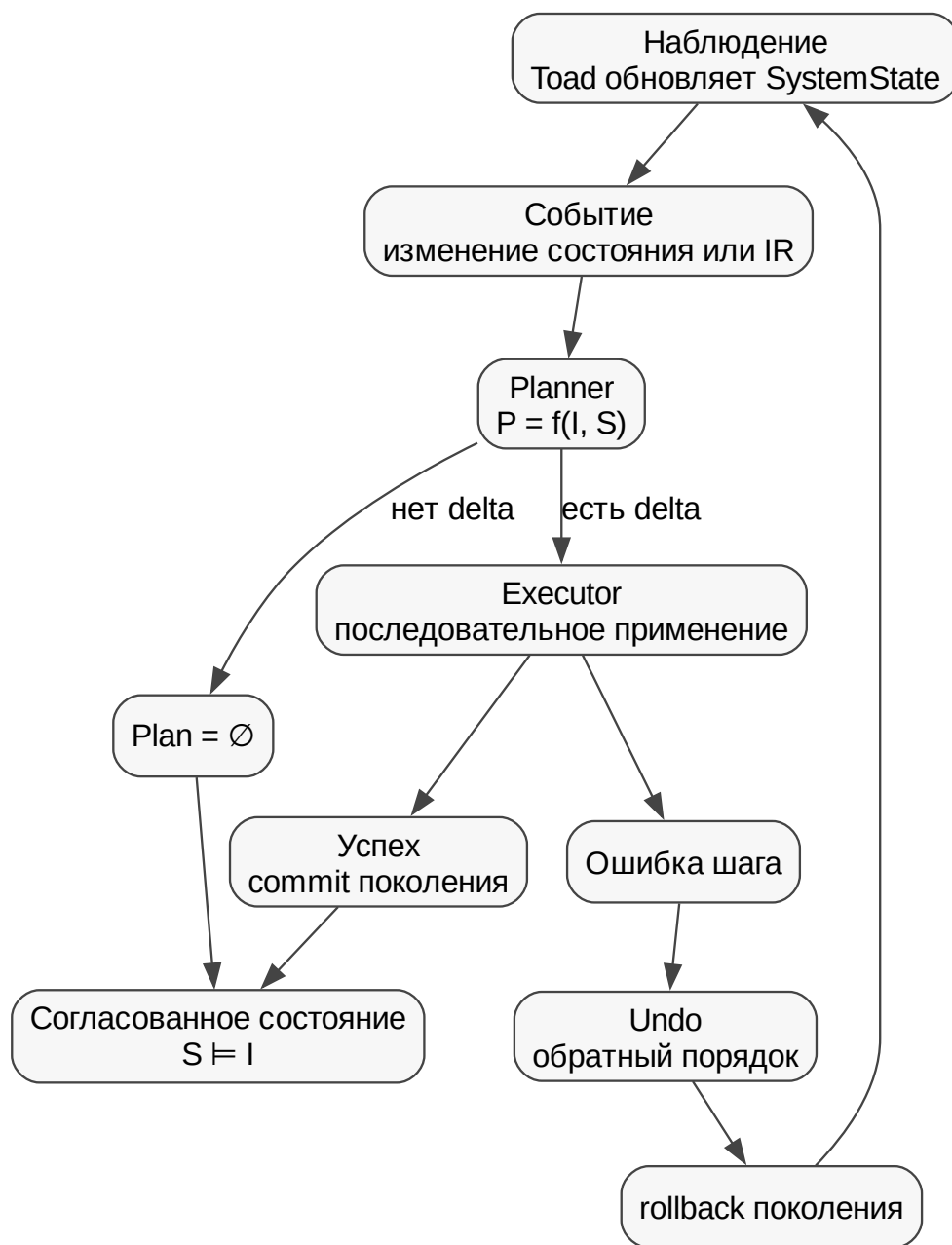


Рис. 2: Цикл наблюдения, планирования и согласования состояния

Такой подход особенно важен для самооптимизации. Например, профиль может активироваться при запуске игры и переключать профиль питания, отключать тяжёлые фоновые модули или добавлять пользовательские пакеты. Когда условие перестаёт быть истинным, следующий снимок состояния приводит к другому плану. Тем самым система реагирует не только на конфигурационные изменения, но и на поведение пользователя.

1.6. Акторы и отказоустойчивость

Runtime FrogOS построен вокруг акторной модели. Актор имеет собственную очередь сообщений и выполняется независимо от других акторов. Он не разделяет изменяемую память с соседями, а получает данные через канал. Это снижает количество скрытых зависимостей и делает сбои более локальными.

Подход напоминает Erlang-модель: рабочие процессы выполняют работу, а Supervisor следит за их жизненным циклом. Если один актор завершился с ошибкой, Supervisor не пытается чинить его внутреннее состояние. Он перезапускает связанную часть цепочки. В FrogOS это особенно важно, потому что Toad, Bridge, Planner и Executor соединены каналами. Если один канал закрыт, часть pipeline нужно создать заново. Стратегии супервизии и принцип “перезапустить процесс, а не восстанавливать его скрытое состояние” соответствуют Erlang/OTP supervision principles [9].

Отказоустойчивость здесь понимается практично. Система не гарантирует, что любой внешний сбой исчезнет сам, но она должна не застрять в неизвестном runtime-состоянии. Supervisor ограничивает частоту перезапусков, чтобы постоянно падающий актор не создавал бесконечный цикл рестартов.

1.7. Транзакционность и компенсация

Операционная система не предоставляет простой общей транзакции для всех изменений. Установка пакета, запись конфигурации, изменение пользователя и перезапуск службы имеют разные механизмы и разные гарантии. Поэтому FrogOS использует модель компенсации: каждый шаг плана сопровождается undo-действием.

Если план применён полностью, поколение фиксируется как успешное. Если один из шагов завершился ошибкой, Executor проходит уже выполненные шаги в обратном порядке и применяет их undo. Это не делает все side effects идеально обратимыми, но даёт системе дисциплину: любой применяемый шаг обязан иметь описанный путь компенсации.

1.8. Проверяемость

Для автономной системы недостаточно проверить только один ручной сценарий. Нужно проверять границы между слоями: DSL должен строить

ожидаемый IR, IR должен сериализоваться без потери смысла, Planner должен быть детерминированным, Executor должен правильно декодировать действия. Поэтому в FrogOS сочетаются обычные unit-тесты, golden-тесты и property-based тестирование.

Property-based подход полезен там, где входное пространство большое. Вместо нескольких вручную выбранных примеров тест генерирует множество допустимых значений и проверяет общее свойство. Для Executor таким свойством является сохранение действия после JSON round-trip. Для DSL и IR таким свойством может быть корректность построения доменных значений через smart constructors и сохранение формы документа после сериализации. В Rust-части для этого используется Serde как стандартный слой сериализации и proptest как библиотека property-based тестирования [12, 13].

II АРХИТЕКТУРА FROGOS

Архитектура FrogOS определяется четырьмя принципами: декларативное намерение отделено от плана исполнения, каждый компонент имеет узкую ответственность, привилегированные операции изолированы за daemon-границей, а любое изменение состояния сопровождается undo-описанием. Этот раздел описывает компоненты системы, их границы и ключевые архитектурные свойства: расширяемость доменов, поколения и аудит, границу привилегий и наблюдаемость runtime.

2.1. Общий конвейер

FrogOS построена как набор слоёв с узкими границами ответственности. Верхний уровень принимает пользовательскую конфигурацию, DSL компилирует её в проверенный IR, Supervisor запускает акторы, Toad наблюдает хост, Planner вычисляет план перехода, а Executor применяет план через backend-адаптеры.

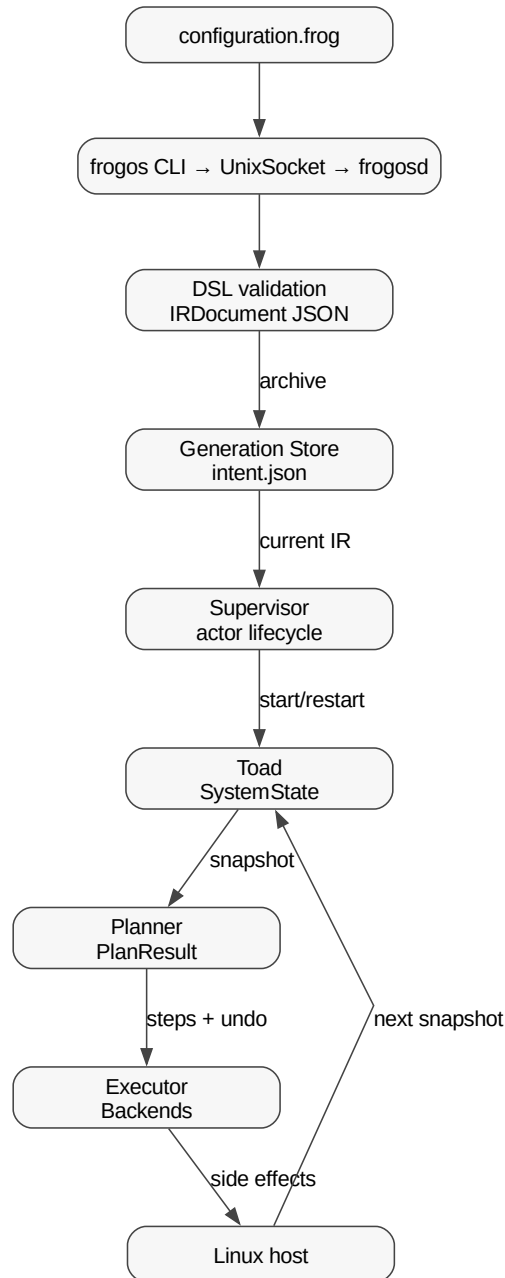


Рис. 3: Архитектура FrogOS

2.2. Организация исходного кода

Исходный код FrogOS ведётся как набор отдельных репозиторий в организации <https://git.cherdak.work/FrogOS>. Архитектурно это соответствует границам системы: каждый слой имеет собственный контракт, тесты и release boundary, а верхний runtime фиксирует совместимые версии через Nix flakes. В текущей структуре проекта выделены следующие

репозитории:

- IR — общий JSON-контракт намерения версии 9: conditions, actions, policies, users, packages, modules и network.
- DSL — Haskell EDSL, smart constructors, semantic validation и compiler path, который создаёт проверенный intent.json.
- Planner — чистое Haskell-ядро планирования, ResourceGraph, delta, ordering, проверка версии IR и protobuf FFI boundary.
- Toad — Rust-слой наблюдения: SystemState и probe-компоненты для процессов, служб, CPU, батареи, пакетов, пользователей, групп, пользовательских пакетов и конфигураций модулей.
- Executor — Rust-исполнитель PlanStep: декодирование DeltaAction, dry-run, backend-traits, dispatch действий, загрузка пакетов через Nix, запись конфигураций модулей в store, управление dinit-модулями и undo выполненных шагов при ошибке.
- Supervisor — actor lifecycle manager, который связывает Toad, Bridge и Planner через watch/mpsc каналы и использует rest-for-one стратегию рестарта.
- FrogOS — верхний workspace для CLI, daemon, protocol и generation store.
- spec и e2e — архитектурные спецификации и интеграционный FHS-контейнер для проверки runtime-сценариев.

На текущем этапе реализованы доменная модель, проверка IR, построение плана, наблюдение, actor-wiring и исполнительный слой для основных системных действий. Executor уже содержит реальные backend-адаптеры: package backend строит пакеты через nix build –out-link в /frogos/store/packages, module install использует nix profile install, config backend записывает конфигурации и dinit service files в /frogos/store/<hash>, а dinit backend вызывает dinitctl enable, dinitctl disable и dinitctl restart. Запланированное развитие включает стабилизацию полного пользовательского пути применения frogos apply: связку daemon, generation store, Supervisor и Executor, фиксацию commit/rollback поколений, расширение проверок безопасности для

привилегированного применения и накопление e2e-сценариев для реальных системных конфигураций.

2.3. Акторная модель и Supervisor

Актор — это независимая единица выполнения с собственным состоянием и входной очередью сообщений. Другие компоненты не изменяют его память напрямую, а отправляют сообщения через канал. Такая модель близка к подходу Erlang: ошибка локализуется в процессе, а отдельный supervisor решает, какие процессы нужно перезапустить [9].

В FrogOS Supervisor не наблюдает систему, не планирует и не исполняет действия. Его задача — собрать pipeline из акторов и удерживать его живым. Он запускает ToadActor, bridge-task, PlannerActor и ExecutorActor. Если Toad падает, перезапускается вся цепочка Toad + Bridge + Planner + Executor. Если Bridge, Planner или Executor падают, перезапускается downstream-часть цепочки. Это соответствует Erlang-стратегии `rest_for_one`: процесс и все процессы после него считаются связанными по ресурсу канала.

Коммуникация внутри runtime построена на каналах Tokio. watch-канал используется для последнего снимка SystemState, mpsc-канал — для очередей сообщений Planner и планов Executor. На границе CLI и daemon используется UnixSocket: локальный сокет позволяет передавать команды привилегированному frogosd, не открывая сетевой интерфейс наружу. Таким образом, внешняя команда остаётся узкой, а привилегированная работа остаётся внутри daemon [11].

2.4. DSL

DSL реализован как Haskell EDSL. Такой подход использует типовую систему языка для раннего отсека структурно невозможных конфигураций. Условие не может быть использовано как действие, действие не может получить стратегию наблюдения, а невалидные значения отсекаются smart constructors. Идея программируемой декларативной конфигурации также используется в GNU Guix, где системные декларации выражаются через Guile Scheme [5, 7].

Листинг 1: Пример модульной серверной конфигурации FrogOS

```
1 serverStack = do
2   Nginx.nginxModule $ do
3     Nginx.enable True
4     Nginx.virtualHost "example.com" $ do
5       NginxVH.httpPort (80 `withFallback` [800])
6       NginxVH.httpsPort (443 `withFallback` [80])
7
8   Fj.forgejoModule $ do
9     Fj.enable True
10    Fj.httpPort (port 3000)
11    Fj.sshPort (port 2222)
12    Fj.domain "git.example.com"
```

DSL не описывает порядок выполнения. Он описывает намерение. Порядок появляется только после наблюдения текущего состояния и построения delta.

2.5. IR

IR является стабильным JSON-контрактом между DSL, Planner и архивом поколений. В текущей архитектуре IR содержит области packages, modules, profiles, users и network. Это позволяет не смешивать пользовательское намерение с runtime-планом.

В практическом виде IR содержит версию схемы, глобальные пакеты, описания модулей, условные профили, пользователей и сетевые ограничения. Версия нужна, чтобы Planner мог явно отвергнуть документ несовместимого формата.

2.6. Toad

Toad отвечает только за наблюдение. Он публикует SystemState и не принимает решений о том, какие действия нужно выполнить. Источниками данных являются таблица процессов, службы, установленные пакеты, пользователи, параметры CPU, состояние батареи и активные приложения.

Важная часть Toad — обработка ObserveStrategy. Если condition задаёт polling-интервал, runtime может создавать таймеры только для реально используемых

условий, не превращая наблюдение в постоянный полный обход всех ресурсов.

2.7. Planner

Planner является stateless Haskell-ядром. Он получает IRDocument и SystemState, проверяет версию IR, выбирает активные профили, строит desired graph, строит current graph, вычисляет delta и возвращает упорядоченные шаги. Ошибки исполнения не являются ошибками Planner: они принадлежат Executor.

2.8. Executor

Executor получает PlanStep, декодирует DeltaAction и маршрутизирует его в соответствующий backend. Для каждого шага есть undo. Это превращает применение плана в ограниченную транзакцию: у каждого прямого шага есть обратный шаг. При сбое Executor проходит уже выполненные действия в обратном порядке.

Ключевая точка расширяемости Executor — enum DeltaAction. Новое управляемое действие добавляется как новый вариант enum и новый обработчик в backend-слое. Сам enum только описывает тип действия и данные; сайд-эффект возникает не при создании значения, а только при интерпретации ExecutorActor.

Листинг 2: Полная модель действий Executor

```
1  #[serde(tag = "type", rename_all = "snake_case")]
2  pub enum DeltaAction {
3      ModuleEnable      { module: ModuleRef, priority: u32 },
4      ModuleDisable     { module: ModuleRef, priority: u32 },
5      ModuleRestart     { module: ModuleRef, priority: u32 },
6      ModuleInstall     { module: ModuleRef },
7      ModuleConfigure   { module: ModuleRef, config_hash: String,
8                          config_json: serde_json::Value },
9      ModuleUnconfigure { module: ModuleRef },
10     PackageInstall    { name: String },
11     PackageRemove     { name: String },
12     UserAdd           { config: UserConfig },
13     UserRemove        { name: String },
14     GroupAdd          { name: String },
15     GroupRemove       { name: String },
16     MembershipAdd     { user: String, group: String },
17     MembershipRemove  { user: String, group: String },
18     UserPackageExpose { user: String, name: String },
19     UserPackageUnexpose { user: String, name: String },
20     PowerProfile       { value: String, priority: u32 },
21     NetworkApply      { allow_ports: Vec<u16> },
22 }
```

2.9. Runtime FrogOS

FrogOS runtime связывает CLI, daemon, generation store, Supervisor и акторы. Пользовательский путь применения начинается в frogos apply, проходит через frogosd, компиляцию DSL в intent.json, запуск Supervisor, планирование и исполнение шагов Executor.

2.10. Расширяемость доменов

Автономная операционная система не может быть закрытой схемой из нескольких жёстко заданных действий. Сегодня она управляет пользователями, пакетами, модулями и профилем питания; позже могут появиться firewall rules, systemd или

dinit units, контейнеры, секреты, графические профили, устройства и политики энергосбережения. Поэтому расширяемость должна быть заложена в архитектуру сразу.

В FrogOS расширение начинается не с добавления новой shell-команды в Executor, а с добавления нового домена в модель намерения. Новый домен должен пройти весь путь: DSL-конструктор, IR-раздел, валидация, наблюдение текущего состояния, планирование delta, действие Executor и backend. Такой путь длиннее, чем быстрая команда в скрипте, но он сохраняет главное свойство системы: каждое действие известно типам, валидатору, планировщику и исполнителю.

Значение действия в IR или PlanStep само по себе не меняет систему. Оно только описывает, что нужно сделать. Реальный side effect возникает в одном месте: в backend, вызванном Executor. Это упрощает тестирование, потому что большинство слоёв можно проверять как преобразование данных.

2.11. Поколения и аудит

Поколение фиксирует результат применения намерения. В FrogOS важнейшим артефактом поколения является intent.json: проверенный IR, который можно прочитать, сравнить с предыдущим поколением и передать Planner. Это отличает поколение от простой директории с файлами. Поколение хранит не только побочный результат, но и объяснение того, какое намерение привело к этому результату.

Аудит поколений нужен для ответа на практические вопросы. Какая конфигурация была активна до сбоя? Какие модули должны были быть включены? Почему Planner попытался установить конкретный пакет? Если intent.json сохранён, ответ можно искать в проверенном документе, а не в неформальной истории команд администратора.

Rollback в такой модели тоже становится понятнее. Откат поколения означает возврат к предыдущему проверенному намерению и повторное согласование системы с ним. Это отличается от простого удаления последних файлов: система должна снова посмотреть на хост, построить план и применить нужные действия.

2.12. Граница привилегий

FrogOS разделяет пользовательский интерфейс и привилегированное применение. CLI принимает команду администратора и передаёт её daemon-слою через локальный UnixSocket. Daemon уже имеет права, необходимые для записи поколений, запуска backend и изменения системного состояния. Межъязыковая граница между Rust и Haskell использует protobuf как компактный типизированный формат обмена структурированными данными [14]. Такое разделение уменьшает поверхность атаки. CLI не должен сам выполнять весь привилегированный pipeline. Он становится клиентом узкого протокола. Runtime может проверять входные команды, контролировать путь к конфигурации, вести журнал применения и централизованно управлять ошибками.

UnixSocket выбран как локальная граница. Он не открывает сетевой порт и естественно вписывается в Unix-модель прав доступа. Это важно для операционной системы: управляющий протокол должен быть доступен локально и контролируемо, а не становиться удалённым API по умолчанию.

2.13. Наблюдаемость runtime

Самолечение невозможно без наблюдаемости. Если система изменила состояние, но не записала, что именно произошло, администратор не сможет понять причину следующего плана. Поэтому FrogOS использует структурированные логи, результаты dry-run, сообщения Store и явные ошибки Planner/Executor.

Наблюдаемость важна и для разработки. При сбое нужно понимать, где именно сломалась цепочка: DSL не собрал IR, daemon не создал поколение, Toad не увидел ресурс, Planner вернул ошибку версии, Executor не смог выполнить backend. Чёткие границы между слоями делают диагностику проще: каждый слой имеет собственный вход, выход и набор ошибок.

2.14. Требования к системе

2.14.1. Функциональные требования

Функциональные требования описывают минимальное поведение, без которого FrogOS не является контуром согласования состояния. Они следуют из

выбранной модели: пользователь задаёт намерение, runtime наблюдает состояние, Planner строит план, Executor применяет план или компенсирует уже выполненные шаги.

- Система должна принимать декларативную конфигурацию через DSL и компилировать её в проверенный IR.
- Система должна выполнять структурную и семантическую валидацию до передачи документа в runtime.
- Система должна хранить проверенный IR как intent.json внутри поколения.
- Toad должен публиковать наблюдаемое состояние в форме SystemState.
- Planner должен строить план как детерминированную функцию от IR и SystemState.
- Planner должен возвращать пустой план, если текущее состояние уже согласовано с намерением.
- Executor должен применять шаги плана последовательно.
- Каждый применяемый шаг должен иметь undo-описание.
- При ошибке применения Executor должен выполнять undo уже завершённых шагов в обратном порядке.
- Runtime должен поддерживать dry-run путь без изменения состояния хоста.
- Система должна поддерживать реакцию на изменение состояния и на смену конфигурации.
- Backend-слой должен изолировать работу с Nix, dinit и системными пользователями от логики Planner.

2.14.2. Нефункциональные требования

Нефункциональные требования фиксируют свойства, которые должны сохраняться при расширении системы новыми доменами. Для FrogOS особенно важны детерминизм, аудит поколений и отделение вычисления плана от

побочных эффектов, потому что ошибка на границе Planner/Executor может привести к неверному изменению хоста.

- Детерминизм: одинаковые входные данные должны давать одинаковый план.
- Воспроизводимость: поколения должны содержать проверенный IR, достаточный для аудита и сравнения.
- Изоляция ответственности: DSL не должен выполнять действия, Planner не должен делать I/O, Executor не должен принимать политические решения.
- Расширяемость: новый домен должен добавляться через новый IR-раздел, валидатор и delta-case, а не через ad-hoc команды.
- Безопасность: привилегированный daemon должен иметь узкий протокол взаимодействия с CLI.
- Наблюдаемость: результат применения, поколение, план и ошибка должны быть доступны для диагностики.
- Идемпотентность: повторное применение согласованного намерения не должно создавать лишних действий.
- Отказоустойчивость: сбой одного шага не должен оставлять runtime без известного состояния поколения.
- Производительность: наблюдение должно использовать стратегии условий и не выполнять полный пересчёт чаще, чем требуется.
- Совместимость версий: Planner должен явно отвергать неподдерживаемую версию IR.

2.15. Сравнительный анализ

2.15.1. Сравнение с NixOS

NixOS предоставляет декларативную конфигурацию системы, атомарные поколения и воспроизводимую сборку пакетов. Его сильная сторона — чистая модель сборки и управление системным профилем [3, 4]. Однако NixOS в

классическом виде не является реактивным runtime-контуром: конфигурация применяется в момент `nixos-rebuild`, а дальнейшее расхождение состояния обычно требует нового запуска команды или внешней автоматики.

FrogOS заимствует идею поколений и воспроизводимости, но переносит центр тяжести с `rebuild`-модели на `reconcile`-модель. IR остаётся намерением, а план зависит от наблюдаемого состояния. Поэтому FrogOS ближе к управляющему контуру: наблюдение, планирование, исполнение и повторное наблюдение.

Принципиальное различие состоит в природе входных данных. Для NixOS главным входом является конфигурация, из которой строится замыкание пакетов и `activation script`. Для FrogOS входом Planner является пара (I, S) : один и тот же IR может дать разные планы при разных снимках состояния. Это делает FrogOS более подходящей для сценариев адаптации: игровой профиль, реакция на батарею, временное отключение модуля, восстановление удалённого пользователя или повторная настройка сервиса после внешнего дрейфа.

NixOS сильнее в области чистой сборки и доказуемого происхождения артефактов. FrogOS в текущей архитектуре не пытается заменить Nix store; напротив, пакетный backend может использовать Nix как исполнитель для `package actions`. Отсюда следует важная инженерная позиция: FrogOS следует рассматривать как реактивный слой управления поверх Linux, а не как альтернативную реализацию функционального пакетного менеджера.

2.15.2. Сравнение с GNU Guix System

GNU Guix System похож на NixOS функциональной моделью пакетов и поколений, но использует GNU Guile как язык конфигурации. Это делает систему программируемой и расширяемой в Lisp-стиле. Guix хорошо подходит для воспроизводимых окружений и свободного программного стека [6, 7, 8].

Отличие FrogOS состоит в явном разделении DSL, IR, наблюдения, планирования и исполнения. Guix-конфигурация описывает системное состояние и сервисы, тогда как FrogOS добавляет слой условий и активных профилей, зависящих от `SystemState`. Таким образом, FrogOS моделирует не только “какой должна быть система”, но и “при каких наблюдениях какая часть намерения активна”.

Guix System использует Shepherd как систему инициализации и управления сервисами. FrogOS в текущей модели ориентируется на `dinit backend` для `module`

lifecycle, но оставляет backend-границу заменяемой. Существенно не то, какой init выбран, а то, что init не принимает политических решений. Он является исполнителем уже вычисленного DeltaAction.

2.15.3. Сравнение с традиционными дистрибутивами

В традиционных дистрибутивах, таких как Debian, Fedora или Arch Linux, конфигурация распределена между пакетным менеджером, systemd/dinit, файлами в /etc, пользовательскими скриптами и ручными действиями администратора. Преимущество такой модели — простота локального вмешательства и зрелость экосистемы. Недостаток — слабая глобальная проверяемость.

FrogOS стремится сделать состояние системы объектом вычисления. Пользователь не редактирует набор независимых императивных шагов, а задаёт намерение. Runtime сам вычисляет delta между current и desired graph.

Инструменты вроде Ansible, Salt или Puppet частично решают проблему дрейфа конфигурации, но обычно работают как внешние системы управления. FrogOS помещает контур наблюдения и согласования внутрь самой операционной модели. Это меняет границу ответственности: система не только исполняет playbook, но и хранит версионированное намерение, наблюдает условия активации и связывает каждый шаг с undo-операцией [15, 16, 17].

2.15.4. Сравнение с самовосстанавливающимися платформами

В Kubernetes и похожих системах контроллеры постоянно приводят кластер к желаемому состоянию. FrogOS концептуально близка к этой controller-модели, но работает на уровне одиночного Linux-хоста: пользователи, пакеты, модули, профили питания и локальные сервисы. Kubernetes управляет контейнерными объектами и сетевыми абстракциями кластера; FrogOS управляет ресурсами операционной системы. Поэтому математическая идея reconcile loop похожа, но пространство ресурсов и требования к безопасности отличаются [10].

Следовательно, FrogOS не является заменой NixOS по глубине пакетной модели. Скорее, она исследует другой слой: реактивное управление уже работающей системой на основе проверенного намерения и наблюдаемого состояния.

III ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1. Структура выполненной реализации

Практическая работа была направлена на построение минимального end-to-end контура FrogOS. Описанная в архитектурной главе схема была разложена на реализуемые подсистемы:

- DSL — Haskell EDSL для описания пользовательского намерения;
- IR — стабильные типы промежуточного представления и JSON-кодирование;
- Toad — Rust-библиотека наблюдения состояния хоста;
- Planner — Haskell-ядро планирования и Rust actor wrapper;
- Executor — Rust-исполнитель действий и undo-операций;
- Supervisor — менеджер жизненного цикла акторов;
- FrogOS — CLI, daemon, протокол и orchestration поколений.

Такое разбиение позволило проверять каждый слой отдельно и затем связать их в общий путь применения конфигурации.

3.2. Реализация DSL и IR

На первом этапе была реализована модель конфигурации, в которой пользователь задаёт пакеты, пользователей, модули и реактивные профили. DSL не формирует команды исполнения напрямую. Он строит проверяемый документ намерения. Внутри DSL намерение разделено на три типа узлов: Condition, Action и Policy. Condition описывает предикат над наблюдаемым состоянием и отвечает на вопрос, когда профиль становится актуальным. Action описывает желаемое изменение системы, которое позже может быть преобразовано в шаг плана. Policy задаёт не действие напрямую, а ограничение или пространство допустимых решений. Например, fallback для порта означает: предпочесть основной вариант, но при конфликте разрешить движку выбрать один из запасных вариантов. Такое разделение делает конфигурацию типобезопасной и сохраняет смысл намерения до runtime-этапа. Стратегия наблюдения via применима только к

Condition, потому что только условие имеет семантику подписки на состояние. Action нельзя использовать как условие, а Policy нельзя свести к простой паре Condition и Action без потери смысла: часть решений, например выбор свободного fallback-порта, может быть корректно принята только во время исполнения.

Листинг 3: Пример реактивного профиля из DSL fixture

```
1 profile "gaming" $  
2   when (processRunning "steam" `via` poll 500) $ do  
3     disable nginx  
4     setPowerProfile Performance
```

Для предотвращения некорректных конфигураций использованы два уровня проверки: типовые ограничения Haskell и семантическая валидация построенного документа. Например, действие не может быть использовано как условие, а стратегия наблюдения применяется только к condition-узлам. После успешной проверки создаётся IRDocument, который может быть сериализован в JSON и сохранён как intent.json поколения.

Этот файл является не журналом команд, а проверенным намерением. Его можно сравнивать между поколениями и повторно передавать Planner без запуска DSL. Desired actions в таком документе содержат priority, потому что Planner разрешает конфликты желаемого состояния по приоритету. Effect actions вроде установки или конфигурации модуля не являются желаемым состоянием и не используют это поле.

Листинг 4: Фрагмент сохранённого intent.json

```
1 {
2   "modules": {
3     "nginx": {
4       "enable": true,
5       "name": "nginx"
6     }
7   },
8   "profiles": [
9     {
10      "actions": [
11        {
12          "module": {
13            "domain": "nginx",
14            "name": "nginx"
15          },
16          "priority": 100,
17          "type": "module_disable"
18        },
19        {
20          "priority": 100,
21          "type": "power_profile",
22          "value": "performance"
23        }
24      ],
25      "condition": {
26        "name": "steam",
27        "observe": {
28          "interval_ms": 500,
29          "type": "poll"
30        },
31        "type": "process_running"
32      },
33      "name": "gaming",
34    }
35  ],
36  "version": 9
```

3.3. Реализация наблюдения состояния

Подсистема Toad выделена как отдельный слой, не принимающий политических решений. Её результат — снимок `SystemState`, содержащий наблюдаемые процессы, приложения, активные модули, установленные пакеты, пользователей, профиль питания и системные метрики.

Практический смысл такого слоя состоит в том, что `Planner` получает не произвольный доступ к хосту, а уже нормализованный снимок состояния. Это сохраняет чистоту планировщика и делает планирование воспроизводимым для одной и той же пары входов (I, S) .

Наблюдение разделено на отдельные `probe`-компоненты. Такой подход важен для расширения: добавление нового источника состояния не должно менять `Planner` или `Executor`. Например, `process probe` отвечает за процессы, `service probe` — за службы, `package probe` — за установленные пакеты, `module config probe` — за конфигурации модулей. `Supervisor` и `Planner` получают уже агрегированный снимок, а не набор несвязанных системных вызовов.

3.4. Реализация Supervisor

`Supervisor` был вынесен в отдельный компонент, потому что жизненный цикл акторов не должен смешиваться с наблюдением, планированием или исполнением. Его задача практическая: создать `ToadActor`, `Bridge`, `PlannerActor` и `ExecutorActor`, связать их каналами и следить, чтобы цепочка не оставалась в разорванном состоянии.

`Bridge` существует потому, что `Toad` и `Planner` говорят на разных представлениях данных. `Toad` публикует внутренний Rust-снимок состояния через `watch` канал. `Bridge` подписывается на изменения, кодирует состояние в `protobuf` и отправляет `PlannerMsg` через `mpsc`. `PlannerActor` получает `NewConfig` и `SystemStateUpdate`, вызывает Haskell FFI и отдаёт `Plan` в канал `Executor`. Каналы `mpsc` используются как `bounded queues` с `backpressure`: при заполнении буфера отправитель ждёт, пока получатель освободит место [11]. `Protobuf` выбран для границы Rust/Haskell, потому что даёт типизированную схему и компактное бинарное представление сообщений [14].

Стратегия рестарта выбрана как `rest_for_one`. Если падает Toad, источник состояния исчезает, поэтому перезапускается вся downstream-цепочка. Если падает Bridge, Planner или Executor, Supervisor пересобирает связанную часть pipeline. Для защиты от бесконечного цикла падений используется лимит интенсивности рестартов: Supervisor хранит историю рестартов в скользящем окне времени и завершает работу ошибкой, если лимит превышен.

3.5. Реализация планирования

Planner реализован как чистое Haskell-ядро. В ходе работы был реализован pipeline: IRDocument и SystemState сначала дают набор активных профилей, затем desired graph и current graph, после чего вычисляется delta и формируется Plan.

Здесь G_d — желаемый граф ресурсов, построенный из IR и активных профилей, G_c — текущий граф ресурсов, построенный из наблюдений, а Δ — набор различий между ними. Полученные действия упорядочиваются по доменам: пользователи, пакеты, модули, профиль питания.

Практическим результатом является отказ от прямого преобразования profile action в команду Executor. Вместо этого система сначала строит desired state, сравнивает его с current state и только затем формирует действия. Это делает повторное применение идемпотентным: если состояние уже соответствует намерению, Planner возвращает пустой план.

3.6. Примеры графов состояния

Планирование удобнее рассматривать на конкретных состояниях. Первый случай — пустой хост: пакеты не установлены, модули не запущены, пользователей нет.



Рис. 4: Пустое исходное состояние системы

Требуемое состояние строится из IR и активных профилей. В примере система должна иметь пользователя, пакеты, активный модуль, сетевые порты и профиль питания.

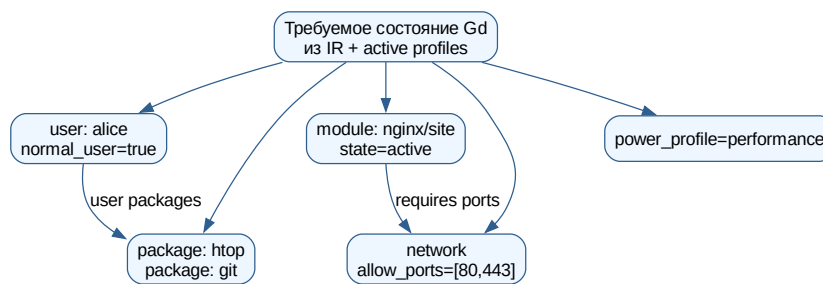


Рис. 5: Требуемое состояние системы

Второй случай — частичный дрейф. Пользователь уже существует, но пакет удалён, модуль остановлен, а профиль питания отличается от требуемого.

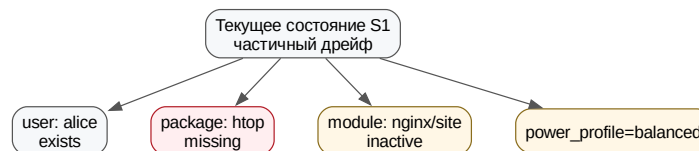


Рис. 6: Текущее состояние системы после дрейфа

Planner сравнивает текущий и требуемый графы и строит план только для отличающихся ресурсов. Уже согласованные ресурсы не попадают в план.

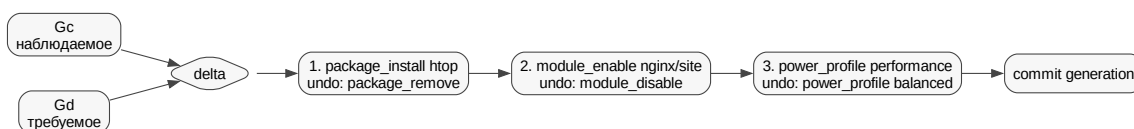


Рис. 7: Пример плана перехода

3.7. Реализация исполнения и отката

Executor принимает не пользовательскую конфигурацию, а уже вычисленные DeltaAction. Исполнение выполняется последовательно, а при ошибке Executor применяет undo для уже выполненных шагов в обратном порядке. В реализации это разделение закреплено тем, что DeltaAction остаётся сериализуемыми данными, а side effect появляется только при вызове backend-метода.

Листинг 5: Маршрутизация DeltaAction в backend внутри dispatch

```
1 async fn dispatch(&self, action: &DeltaAction)
2     -> Result<(), ExecutorError>
3 {
4     match action {
5         DeltaAction::ModuleEnable { module, .. } =>
6             self.backends.dinit.enable_module(&module.name).await,
7         DeltaAction::ModuleDisable { module, .. } =>
8             self.backends.dinit.disable_module(&module.name).await,
9         DeltaAction::PackageInstall { name } =>
10            self.backends.packages.install_package(name).await,
11         DeltaAction::PackageRemove { name } =>
12            self.backends.packages.remove_package(name).await,
13         DeltaAction::UserAdd { config } =>
14            self.backends.system.add_user(config.clone()).await,
15         DeltaAction::NetworkApply { allow_ports } =>
16            self.backends.network.apply_ports(allow_ports).await,
17         // ... остальные варианты аналогично
18     }
19 }
```

Метод `dispatch` является единственной точкой, где `DeltaAction` порождает side effect. Само значение `enum` — это данные. Изменение системы происходит только здесь, при передаче команды соответствующему backend-трейту. Поэтому все остальные слои — `Planner`, валидатор, тесты — работают с `DeltaAction` как с чистыми данными без риска изменить хост.

3.8. Проверка свойств

Для DSL и IR использованы golden-тесты и round-trip проверки JSON. Они фиксируют форму выходного IR и проверяют, что сериализация не меняет смысл документа. Для доменных значений используются генераторы `Arbitrary`: например, имена процессов, polling-интервалы, CPU threshold и battery percent генерируются с учётом smart constructors, поэтому тесты проверяют только допустимое пространство значений.

В Executor применено property-based тестирование DeltaAction. Тесты генерируют разные варианты enum, сериализуют их в JSON и декодируют обратно. Это важно, потому что Planner и Executor связаны JSON-представлением действий внутри PlanStep.

Листинг 6: Property-тест DeltaAction: round-trip через JSON

```
1 proptest! {  
2     #[test]  
3     fn serde_roundtrip_preserves_delta_action_state(  
4         action in delta_action()  
5     ) {  
6         let encoded = serde_json::to_value(&action)  
7             .map_err(prop_err)?;  
8         let decoded = serde_json::from_value:::<DeltaAction>(encoded)  
9             .map_err(prop_err)?;  
10  
11         prop_assert_eq!(decoded, action);  
12     }  
13 }
```

Такой тест защищает границу между планированием и исполнением: если новый вариант действия добавлен в enum, но сломан JSON-контракт, ошибка проявится до запуска реального backend.

3.9. Реализация dry-run пути

Для безопасной проверки был реализован dry-run путь применения конфигурации. Команда верхнего уровня передаёт конфигурацию daemon-слою, после чего frogos-dsl-compile создаёт поколение в изолированном временном каталоге. Реальное состояние основного хоста при этом не изменяется. При этом исполнительный слой уже имеет реальные backend-операции для изолированной среды: Nix-загрузка пакетов, запись конфигураций в /frogos/store и управление dinit-модулями через dinitctl.

Практический pipeline проходит через DSL, ValidatedIR, intent.json, SystemState, PlanResult и DeltaAction. Каждый этап имеет отдельную границу ответственности и отдельный формат данных.

Листинг 7: Пример вывода dry-run применения

```
1 $ frogos apply --dry-run ./examples/gaming.hs
2 generation: /tmp/frogos-generations/000004
3 intent:      /tmp/frogos-generations/000004/intent.json
4 observe:     SystemState { processes: ["steam"], modules: ["nginx"] }
5 plan:
6   1. module_disable module=nginx priority=100 undo=module_enable
7   2. power_profile value=performance priority=100
   ↪   undo=power_profile
8 result: dry-run, host state unchanged
```

3.10. End-to-end окружение

Для проверки связки компонентов подготовлен отдельный путь e2e через Nix flake. Он создаёт FHS-контейнер frogos-e2e, в который прокидываются собранные пакеты FrogOS и DSL, а также инструменты, нужные для системного сценария: nix, dinit, forgejo, jq, procs и базовые shell-утилиты. Внутри контейнера создаются временные файловые системы для /frogos, /etc/dinit, /etc/frogos и /run. Это позволяет проверять daemon socket, generation store, dinit service directory, пользовательскую конфигурацию, загрузку пакетов через Nix и запуск dinit-модулей без изменения основной системы разработчика.

Листинг 8: Запуск интеграционного FHS-окружения

```
1 $ nix run git+https://git.cherdak.work/FrogOS/e2e
2
3 === FrogOS interactive environment ===
4   frogosd -> /run/frogosd.sock
5   dinit   -> $XDG_RUNTIME_DIR/dinitctl
6   config  -> /etc/frogos/Main.hs
```

В этом окружении можно запускать верхний runtime, проверять доступность frogosd, компилировать /etc/frogos/Main.hs через DSL, создавать поколения в /frogos/generations, скачивать пакеты в изолированный store и активировать dinit-модули из временного service directory. Именно этот FHS-контейнер используется как практическая граница готовности: компонент считается

пригодным для runtime-интеграции, когда он собирается через flake, попадает в контейнер и работает в условиях, близких к обычной Linux иерархии каталогов.

Листинг 9: Фрагмент вывода тестов реализации

```
1 $ cargo test -p executor
2 test empty_plan_sends_commit ... ok
3 test step_fails_executes_undo_in_reverse_and_sends_rollback ... ok
4 test dispatches_module_configure ... ok
5 test dispatches_network_apply ... ok
6 test serde_roundtrip_preserves_delta_action_state ... ok
```

В CI и локальном e2e-пути проверки запускаются в окружении, собранном через Nix flakes. Это снижает риск того, что тесты проходят только на машине разработчика из-за неявно установленных библиотек или старых артефактов сборки.

Таким образом, выполненная часть проекта подтверждает основную архитектурную гипотезу: FrogOS можно строить как реактивную систему, где декларативное намерение, наблюдаемое состояние и план перехода разделены жёсткими границами.

3.11. Инструменты разработки

- **Git** — использовался для контроля версий, разделения работы по веткам и фиксации архитектурных изменений маленькими проверяемыми шагами. Для проекта с несколькими репозиториями это особенно важно: изменение IR часто требует синхронных изменений в DSL, Planner и Executor, поэтому история коммитов должна показывать, какая граница контракта изменилась и почему.
- **Self-hosted Forgejo** — использовался как собственная платформа размещения репозитория. Это снижает зависимость от внешних сервисов и даёт контроль над доступом к внутренним компонентам FrogOS. Для проекта операционной системы это важно, потому что инфраструктура разработки тоже является частью доверенной цепочки.
- **Gitea runner** — использовался для CI. Он запускал проверки в окружениях, созданными Nix flakes. CI нужен не только для тестов, но и для проверки

того, что компоненты собираются в чистом окружении, без скрытых локальных зависимостей разработчика.

- **Nix** — использовался для управления зависимостями, dev shell и воспроизводимых сборок. Через flakes фиксируются версии Rust toolchain, Haskell packages, форматтеров, protobuf-инструментов и внешних репозиториях. Nix также упрощает FFI-сборку: Planner может экспортировать Haskell library, а Rust planner-actor получает путь к ней через окружение.
- **FHS-контейнер** — использовался для ручного тестирования тех частей, которым нужен более обычный Linux filesystem layout. Чистое Nix окружение удобно для сборки, но не всегда похоже на реальный хост. FHS контейнер позволяет проверять пути вроде /etc, service files, symlink, dinit socket и сценарии, где backend взаимодействует с системой как обычная Linux-программа.
- **STAN** — использовался для статического анализа Haskell-кода. Его задача — находить рискованные места до runtime: неполные pattern match, опасные функции, неудачные идиомы и потенциально хрупкие участки чистой логики. Для DSL, IR и Planner это важно, потому что ошибка в чистом коде может породить неверный план.
- **Clippy** — использовался для статического анализа Rust-кода с режимом -D warnings. Он помогает держать runtime-часть аккуратной: ловит лишние аллокации, сомнительные API-границы, ошибки владения, неудачные async-паттерны и места, где код можно сделать проще.

ЗАКЛЮЧЕНИЕ

В результате работы FrogOS была рассмотрена как реактивная система управления состоянием Linux-хоста. Основной вывод состоит в том, что самолечение и самооптимизацию удобнее строить не как набор отдельных скриптов, а как замкнутый контур: проверенное намерение, наблюдаемый снимок системы, детерминированный план перехода и контролируемое исполнение с undo-действиями.

В работе сформирована архитектура такого контура. DSL отделяет пользовательское описание от runtime, IR фиксирует стабильный контракт намерения, Toad приводит состояние хоста к единому SystemState, Planner строит delta между желаемым и наблюдаемым состоянием, Executor применяет PlanStep через backend-адаптеры, а Supervisor удерживает actor pipeline в рабочем состоянии. Такое разделение позволяет проверять большинство решений как преобразование данных, а реальные side effects оставлять в узком исполнительном слое.

Практическая часть показала, что выбранная модель применима к рабочим сценариям FrogOS. Реализованы доменная модель IR версии 9, Haskell DSL с валидацией, графовое планирование, protobuf FFI между Haskell и Rust, наблюдение SystemState, исполнительные backend-операции для Nix, dinit, конфигураций модулей, пользователей, пользовательских пакетов и network, а также rollback выполненных шагов при ошибке. Для проверки связки компонентов подготовлен e2e FHS-контейнер, который запускает runtime в изолированной Linux иерархии каталогов и позволяет проверять пакеты, dinit socket, generation store и daemon socket без изменения основной системы.

Формально итоговая модель может быть представлена так:

$$I \rightarrow IR, \quad (IR, S_t^{obs}) \rightarrow (G_d, G_c) \rightarrow P_t \rightarrow S_{t+1}^{obs}.$$

Эта схема подчёркивает важное ограничение: FrogOS работает не с абсолютной истиной о машине, а с наблюдаемым состоянием, собранным probe-компонентами. Поэтому каждый цикл завершается новым наблюдением, а не предположением, что исполнение автоматически привело систему к требуемому состоянию.

Дальнейшее развитие проекта связано со стабилизацией полного пользовательского пути применения frogos apply, фиксацией commit/rollback

поколений, расширением набора наблюдаемых ресурсов, усилением проверок безопасности для привилегированного daemon и накоплением end-to-end сценариев для реальных системных конфигураций. Тем не менее уже реализованная часть подтверждает основную гипотезу курсовой работы: автономное управление состоянием можно строить как строгий reconciliation loop поверх обычного Linux-хоста, не смешивая намерение, наблюдение, планирование и исполнение в один неуправляемый слой.

БИБЛИОГРАФИЯ

- [1] Kephart J. O., Chess D. M. *The Vision of Autonomic Computing*. Computer, 36(1), 2003, pp. 41–50. DOI: <https://doi.org/10.1109/MC.2003.1160055>.
- [2] IBM. *An Architectural Blueprint for Autonomic Computing*. White Paper, Third Edition, 2005. URL: <https://users.cs.fiu.edu/~sadjadi/Teaching/Autonomic%20Grid%20Computing/CIS-6612-Summer-2006/AC-Blueprint-WhitePaper-V7.pdf>.
- [3] NixOS Project. *NixOS Manual*. URL: <https://nixos.org/manual/nixos/stable/>
- [4] NixOS Project. *Nix Pills: Why You Should Give it a Try*. URL: <https://nixos.org/guides/nix-pills/01-why-you-should-give-it-a-try.html>
- [5] GNU Project. *GNU Guix Reference Manual: Using the Configuration System*. URL: https://guix.gnu.org/manual/1.5.0/en/html_node/Using-the-Configuration-System.html
- [6] GNU Project. *GNU Guix transactional package manager and distribution*. URL: <https://guix.gnu.org/>
- [7] GNU Project. *GNU Guile Manual*. URL: <https://www.gnu.org/software/guile/manual/>
- [8] GNU Project. *The GNU Operating System*. URL: <https://www.gnu.org/gnu/gnu.html>
- [9] Erlang/OTP. *Erlang/OTP System Documentation: Supervision Principles*. URL: https://www.erlang.org/doc/system/sup_princ.html
- [10] Kubernetes. *Kubernetes Documentation: Controllers*. URL: <https://kubernetes.io/docs/concepts/architecture/controller/>
- [11] Tokio. *Tokio mpsc channel documentation*. URL: <https://docs.rs/tokio/latest/tokio/sync/mpsc/fn.channel.html>

- [12] Serde Project. *Serde: Rust serialization framework*. URL: <https://serde.rs/>
- [13] proptest-rs. *Proptest Book*. URL: <https://altsysrq.github.io/proptest-book/>
- [14] Google. *Protocol Buffers Documentation*. URL: <https://protobuf.dev/>
- [15] Red Hat. *Ansible Documentation*. URL: <https://docs.ansible.com/>
- [16] Salt Project. *Salt Project Documentation*. URL: <https://docs.saltproject.io/>
- [17] Puppet. *Puppet Documentation*. URL: <https://www.puppet.com/docs>